

Financial Time-Series MLOps

Predicting Next-Day Market-Open Direction
for NSE F&O Stocks Using OHLCV and News Sentiment

Student Name

Roll Number

MLOps Course End-Term Project

August 23, 2026

Contents

1	Problem Statement	2
2	Dataset Description	2
2.1	Stock Universe	2
2.2	OHLCV Price Data	2
2.3	News Headlines	3
2.4	Data Contracts	3
3	System Architecture	3
3.1	High-Level Data Flow	4
3.2	Orchestration	4
3.3	Data and Model Versioning	4
4	Data Pipeline	4
4.1	Scrapers	4
4.2	PySpark Processing	5
4.3	Airflow Inference DAG	5
5	Model Development	5
6	Deployment Strategy	5
7	Monitoring Strategy	5
8	CI/CD Implementation	5
9	Results and Discussion	6
10	Challenges Faced	6
11	Future Improvements	6

1 Problem Statement

This project builds an end-to-end MLOps pipeline to predict the **next-day market-open price direction** (Bullish / Bearish / Neutral) for 50 NSE Futures & Options (F&O) constituent stocks.

The prediction target is a three-class variable derived from the return between the next day's 09:15 AM opening price and its closing price. A return exceeding +0.3% is labelled *Positive*, below -0.3% is *Negative*, and anything in between is *Neutral*.

The system ingests two complementary data streams:

1. **OHLCV price data** — 15-minute intraday bars from TradingView, aggregated into daily candles with rolling technical indicators.
2. **Financial news headlines** — scraped from Groww using Crawl4AI, enriched with FinBERT (ProsusAI/finbert) sentiment probabilities and 768-dimensional CLS embeddings, then PCA-reduced and aggregated with exponential-decay rolling windows.

The merged feature set (95 features) feeds an XGBoost / LightGBM classifier trained with TimeSeriesSplit cross-validation and tracked via MLflow. The best model is served through a containerised FastAPI endpoint and monitored with Prometheus metrics.

Minimum expected scope covered.

- Automated pipeline using **Airflow** (inference and training DAGs).
- Additional data-engineering tool: **PySpark** (headline cleaning, pre-open merge).
- Data preprocessing workflow (scraping → Spark cleaning → FinBERT → feature engineering).
- Training and comparison of **two models** (XGBoost and LightGBM).

2 Dataset Description

All data is stored as DVC-tracked CSV files in the repository's `data/` directory.

2.1 Stock Universe

The universe comprises **50 NSE F&O stocks** listed in `data/stock_list.csv`. Each row maps a company name (`Stock_name`) to its TradingView identifier, Groww slug, and NSE symbol.

2.2 OHLCV Price Data

Table 1: OHLCV data files		
File	Granularity	Description
<code>merged_ohlc_15min.csv</code>	15-minute	Intraday bars for all 50 stocks. Columns: <code>symbol</code> , <code>datetime</code> , <code>open</code> , <code>high</code> , <code>low</code> , <code>close</code> , <code>volume</code> .
<code>ohlc_data/<stock>.csv</code>	15-min / daily	Per-stock raw files (daily and 15-min), merged into the above.
<code>preopen_csv/</code>	Point-in-time	NSE F&O pre-open auction data, one dated CSV per trading day.

OHLCV data is fetched via the TradingView data feed (`tvDatafeed` library). Each stock has up to 10 000 historical 15-minute bars (~2 years). The scraper supports incremental updates, fetching only bars newer than the latest timestamp in the existing CSV.

2.3 News Headlines

Table 2: Headline data files

File	Description
stock_news/headlines.csv	Raw scraped headlines (Crawl4AI from Groww). Columns: headline_id, symbol, published_at, source, headline, article_url, scraped_at.
headlines.csv	Cleaned and deduplicated headlines (PySpark output, DVC-tracked).
headlines_enriched.csv	FinBERT-enriched: positive_prob, negative_prob, neutral_prob, sentiment_score, sentiment_label.
headlines_enriched.embeddings.npy	768-d FinBERT CLS embeddings (NumPy binary).
news_features.csv	PCA-reduced (50-d) + 7-day rolling aggregated sentiment features per (symbol, date).

2.4 Data Contracts

All inter-role data exchange is governed by dataclass schemas defined in `shared/schemas/data_contract.py`:

```
1 @dataclass
2 class PriceRecord:
3     symbol: str          # e.g. "Reliance Industries"
4     date: str            # ISO-8601 date
5     open: float
6     high: float
7     low: float
8     close: float
9     volume: int
10
11 @dataclass
12 class HeadlineRecord:
13     headline_id: str
14     symbol: str
15     published_at: str # ISO-8601 datetime
16     source: str
17     headline: str
18     author: Optional[str] = None
19     body_snippet: Optional[str] = None
```

Listing 1: Core data-contract schemas

3 System Architecture

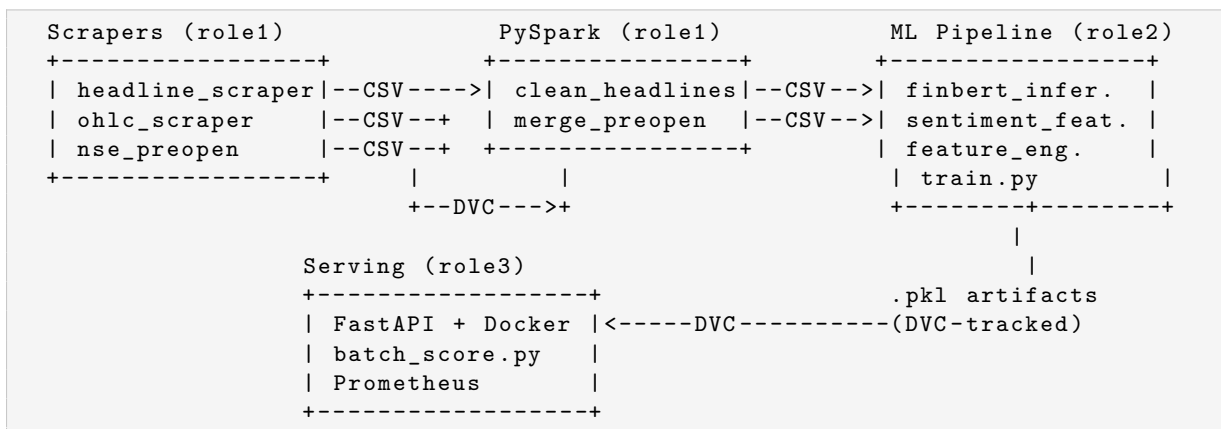
The project is organised into three roles, each with its own directory, dependencies, and Airflow DAGs:

Table 3: Role decomposition

Role	Directory	Responsibility
1	role1_data_engineering/	Scraping, PySpark cleaning, Airflow orchestration, DVC data versioning
2	role2_ml_modeling/	Feature engineering, model training (XGBoost / LightGBM), MLflow experiment tracking
3	role3_mlops_devops/	FastAPI serving, Docker containerisation, Prometheus monitoring, CI/CD

Shared configuration lives in `shared/config.py` (environment-variable driven, timezone-aware) and data contracts in `shared/schemas/data_contract.py`.

3.1 High-Level Data Flow



Listing 2: End-to-end data flow

3.2 Orchestration

Two Airflow DAGs drive the system:

1. **Inference DAG** (`financial_ts_inference_pipeline`) — daily at 08:30 IST: scrape → Spark cleanup → Docker inference → DVC push.
2. **Training DAG** (`financial_ts_training_pipeline`) — manual or periodic: FinBERT → sentiment features → feature engineering → parallel XGBoost/LightGBM sweeps → DVC push models.

3.3 Data and Model Versioning

DVC (Data Version Control) tracks all large artefacts:

- `data/headlines.csv.dvc`, `data/merged_ohlcv_15min.csv.dvc`
- `models/*.pkl.dvc` (XGBoost, LightGBM, PCA, encoder, feature columns)

The DVC remote is Google Drive (service-account authentication). Model versions are git-tagged (`model-v1`, `model-v2`).

4 Data Pipeline

4.1 Scrapers

Four modules ingest raw data daily. All support incremental updates.

Module	Source	Output	Notes
headline_scraper	Groww (Crawl4AI)	stock_news/headlines.csv	Async headless browser; MD5-based <code>headline_id</code> for dedup
ohlc_scraper	TradingView	merged_ohlc_15min.csv	Up to 10k 15-min bars/stock; per-stock CSVs merged
nse_preopen_scraper	NSE API	preopen_csv/*.csv	F&O pre-open auction; exponential-backoff retries
finbert_inference	ProsusAI/finbert	enriched CSV + .npy	Batched sentiment probs + 768-d CLS embeddings; $s = (p_+ - p_-)(1 - p_0)$

4.2 PySpark Processing

Two PySpark jobs run in `local[*]` mode. **clean_headlines** applies a UDF (lowercase, strip HTML, collapse whitespace), deduplicates by `headline_id` via a window function, and writes `data/headlines.csv`. **merge_preopen** maps NSE tickers to stock names via a Spark join on `stock_list.csv`, creates synthetic 09:15 OHLCV bars from the pre-open data, deduplicates, and appends to `merged_ohlc_15min.csv`. A third job, **trim_for_inference**, filters the full CSVs to the minimal date window (45-day OHLCV, 10-day headlines) needed by the Docker inference container.

4.3 Airflow Inference DAG

The DAG `financial_ts_inference_pipeline` runs daily at 08:30 IST with seven tasks: scrape headlines + OHLCV (parallel) → scrape pre-open → Spark clean + merge (parallel) → Spark trim → Docker inference (`batch_score.py`, trimmed data mounted read-only) → DVC push. Each task runs its module via `subprocess.run()` for import isolation. Setting `SKIP_SCRAPERS=1` disables live scraping for testing.

5 Model Development

This section will detail the FinBERT → PCA → sentiment aggregation pipeline, OHLCV technical-indicator feature engineering, XGBoost and LightGBM training with TimeSeriesSplit cross-validation, and MLflow experiment tracking. To be completed by the ML modelling team member.

6 Deployment Strategy

This section will describe the Docker containerisation strategy, FastAPI serving layer, batch scoring integration with Airflow, and the Postgres prediction store. To be completed by the MLOps team member.

7 Monitoring Strategy

This section will describe the Prometheus-based monitoring setup, the four custom metrics exposed at `/metrics`, and example PromQL queries. To be completed by the MLOps team member.

8 CI/CD Implementation

This section will describe the GitHub Actions CI/CD pipeline: test job (pytest across all three roles), Docker image build on main branch merges, and planned enhancements (container boot test, `/health` probe). To be completed by the MLOps team member.

9 Results and Discussion

This section will present the cross-validation results for both XGBoost (best $F1 = 0.6631$) and LightGBM (best $F1 = 0.6572$), per-class metrics, feature importance analysis, and discussion. To be completed by the ML modelling team member.

10 Challenges Faced

11 Future Improvements

This section will outline planned enhancements: data-drift detection, a prediction frontend, Alembic migrations, additional data sources, and Prometheus Pushgateway integration for batch-job metrics. To be completed collaboratively.